

PROBLEM STATEMENT

Intelligent Emergency Response & Resource Optimization Platform

Design a backend system capable of managing and optimizing emergency response operations for a large-scale country-wide disaster management network.

The platform must continuously receive emergency incidents from multiple regions while coordinating available emergency resources such as ambulances, hospitals, rescue teams, helicopters, and emergency operation centers.

Each emergency incident has its own characteristics, including but not limited to:

- ▶ Geographic location
- ▶ Severity level
- ▶ Number of affected people
- ▶ Time sensitivity
- ▶ Resource requirements
- ▶ Environmental condition

Resources also have dynamic properties, such as:

- ▶ Current location
- ▶ Availability
- ▶ Capacity
- ▶ Estimated travel time
- ▶ Operational constraints
- ▶ Temporary failures or maintenance

During execution, the environment is expected to change continuously. New emergency requests may arrive at any moment, roads may become unavailable, hospitals may reach full capacity, vehicles may fail, communication delays may occur, and resource availability may change unexpectedly.

Your proposed system should continuously make near-optimal, explainable, and operationally feasible decisions while adapting to these changing conditions in near real time.

The system should be designed for very high throughput and should remain reliable, scalable, fault tolerant, and maintainable under heavy load.

YOUR OBJECTIVE

Your submission should demonstrate how your backend architecture and decision-making engine would:

- ▶ Accept and process a continuous stream of emergency events.
- ▶ Prioritize incidents intelligently.
- ▶ Allocate the most appropriate resources.
- ▶ Continuously re-optimize decisions whenever the environment changes.
- ▶ Minimize overall response time.
- ▶ Maximize resource utilization.
- ▶ Prevent resource conflicts.
- ▶ Remain operational even when parts of the system become unavailable.
- ▶ Scale efficiently as the number of users, incidents, and resources increases.

FUNCTIONAL EXPECTATIONS

Your proposed solution should clearly explain:

- ▶ Overall system architecture
- ▶ Data flow
- ▶ Service interactions
- ▶ Decision-making strategy
- ▶ Optimization approach
- ▶ Failure recovery strategy
- ▶ Scalability strategy
- ▶ Performance considerations
- ▶ Security considerations
- ▶ Monitoring and observability
- ▶ Database design
- ▶ Caching strategy
- ▶ Event processing strategy

You are expected to justify every important architectural and technical decision.